

FINAL YEAR PROJECT · VIVA PREPARATION

Movera

AI-Powered Intelligent Physiotherapy Assessment and Rehabilitation Monitoring System

REPOSITORY	<code>physio-ai-platform</code> — npm workspace monorepo, v1.0.0
ARCHITECTURE	React SPA → NestJS API → PostgreSQL, with a FastAPI computer-vision sidecar
SCALE	~24,000 lines of application code · 305 automated tests · 19 database models
DEPLOYMENT	Frontend on Vercel; backend, CV service, database and TLS proxy on one AWS EC2 instance
DOCUMENT	Technical summary and examiner Q&A bank, 12 September 2026

HOW TO USE THIS DOCUMENT

Parts I–XI are the technical narrative — read them once to rebuild the whole system in your head. **Part XII is the Q&A bank:** twenty-six questions an examiner is likely to ask, each with a defensible answer grounded in code that actually exists in this repository. The appendix is a one-page numbers sheet to skim immediately before you walk in.

— Contents

1. The problem and the objectives — what the system is for
2. Provenance: four prototypes, one system — what was inherited and what was built
3. System architecture — three services and why they are separate
4. Technology choices and their justification
5. The data model — 19 entities, and the ones that matter
6. The computer-vision pipeline — pixels to repetitions
7. Repetition counting — the state machine, and why it exists
8. Data-driven rules and clinical versioning
9. The scoring model — explainable, not a black box
10. The live session, end to end
11. The trust model and security design
12. Frontend architecture and the design system
13. Testing strategy
14. Deployment architecture
15. Limitations and future work
16. **Examiner Q&A bank** — 26 questions with model answers

Appendix — Numbers at a glance.

THE THIRTY-SECOND ANSWER

If an examiner opens with “tell us what you built”, this is the shape of the answer:

Movera lets a physiotherapist prescribe exercises to a patient, and lets the patient perform them at home in front of a webcam. The browser captures frames and streams them to a pose-estimation service, which detects seventeen body keypoints, computes joint angles, and runs a state machine that decides when a genuine repetition has been completed and whether it was performed correctly. Every counted repetition is written to the database by the NestJS backend — never by the browser — and at the end of the session the backend computes the report from its own persisted rows. The therapist sees the result on their dashboard. The clinical thresholds are database rows rather than code, and every session records which version of those rules judged it.

I. The problem and the objectives

Rehabilitation succeeds or fails on what happens between appointments — and that is exactly the part nobody can see.

The clinical problem

A physiotherapy programme is mostly unsupervised. A patient is shown an exercise in a twenty-minute appointment, then performs it alone, at home, perhaps eighty times before the next visit. Two failures follow from this:

- **Adherence is invisible.** The therapist has only the patient's recollection of how many sessions they actually did. Self-report is unreliable and systematically optimistic.
- **Form degrades unobserved.** A squat performed with insufficient depth, or a shoulder abduction performed by hitching the shoulder rather than raising the arm, is not merely useless — it reinforces the compensation pattern the therapy is meant to remove. Nobody corrects it because nobody sees it.

Objectives

OBJECTIVE	HOW THE SYSTEM MEETS IT
Objective measurement	Joint angles computed from pose estimation, not from patient self-report. A repetition is counted by a state machine observing the angle trajectory.
Real-time correction	Form faults are detected per frame and surfaced to the patient <i>during</i> the movement, while the correction can still be applied.
Remote monitoring	The therapist's dashboard shows every session each patient performed, with scores, error patterns and range-of-motion trends.
Clinical configurability	Thresholds live in the database. A physiotherapist can retune an exercise without a code change or a redeployment.
Auditability	Every session stores the rule-config version that judged it, so a historical report can always be reinterpreted against the criteria actually in force at the time.

Scope — what it deliberately is not

- **Not a diagnostic device.** It measures performance against a prescription; it does not diagnose pathology. Every patient-facing surface carries a prototype disclaimer.
- **Not a replacement for the therapist.** The therapist prescribes, reviews and adjusts; the system is instrumentation between appointments.
- **Not clinically validated.** The thresholds are drawn from the project's prototype work and general biomechanics, not from a trial. This is stated plainly rather than hidden — see Part XV.

The five supported exercises

EXERCISE	GOAL TYPE	PRIMARY MEASUREMENT AND CHARACTERISTIC FAULT
squat	Reps	Knee flexion angle, both legs averaged. Faults: insufficient depth, trunk lean, knee alignment (valgus).
bicep-curl	Reps	Elbow angle, most-flexed side drives the count. Faults: elbow drift, incomplete range of motion.
shoulder-abduction	Reps	Shoulder abduction angle. Faults: shoulder hitch, trunk compensation.
straight-leg-raise	Reps	Hip flexion with a knee-extension constraint. Fault: knee flexed during the raise.
static-posture	Hold	Counts nothing. Accumulates seconds held in correct alignment. Faults: shoulder, hip and spine alignment.

WHY THIS MATTERS ARCHITECTURALLY

`static-posture` is not a special case bolted on. The distinction is carried as an `ExerciseGoalType` enum (`REPS` or `HOLD`) that flows from the database through the analyzer context into the pose service, and splits again at report generation. The same `BaseExerciseAnalyzer` class handles both; only the completion driver differs (`MovementMachine` versus `HoldTracker`).

II. Provenance: four prototypes, one system

This was a consolidation, not a greenfield build — and being straightforward about that is a strength in a viva, not a weakness.

Four partial implementations existed on disk before the current system was written. Understanding what each contributed, and what was rejected, is the single most likely source of a difficult examiner question.

PRIOR ARTEFACT	WHAT IT WAS	DISPOSITION
FYP/.../OpenCV-Unity-To-Build-3DHands	Unity + UDP hand-tracking demo	Discarded, and attributed. It is a fork of a public repository; the team's own additions amounted to a gesture <code>if/elif</code> ladder, a camera-index change and a string suffix. Claiming it as original work would be the worst possible answer in a viva.
FYP/.../physio_backend	FastAPI + YOLOv8 + SQLite	Harvested. Its <code>analysis/</code> modules — angle mathematics, five exercise analyzers, name-addressed joints — are the most valuable prior code and survive into <code>services/pose-service</code> . The thresholds were preserved exactly; the transition logic was replaced.
backend/	Express + MongoDB	Superseded. A document store is the wrong shape for data that is fundamentally relational (a session belongs to an assignment belongs to a plan belongs to a therapist–patient link).
frontend/	Create-React-App running MediaPipe in-browser	Superseded, one idea kept. The idea worth keeping was that <i>the camera stays in the browser</i> . What was rejected was that the browser also counted the repetitions and posted its own score.

THE SINGLE MOST IMPORTANT INHERITED DEFECT

The CRA prototype's React page posted a literal `accuracy: 80` to its backend. The score was a hard-coded constant dressed up as a measurement. Correcting that — making the backend authoritative over every number in the report — is the architectural spine of the current system, and it is the story to tell if asked “what is the most significant thing you changed?”

The YOLO question

Be ready for this one, because it has an awkward history. The project brief specified no YOLO. It later emerged that a YOLOv8 dependency was *already embedded* in the inherited FastAPI prototype, unknown to the team. The current system uses `YOLOv8n-pose` deliberately and documents it.

Two consequences worth being able to state without hesitation:

- **Licensing.** `ultralytics` is **AGPL-3.0** — a copyleft licence that is triggered by *network use*, not just distribution. Hosting this service publicly obliges you to offer its source under the same terms. Acceptable for academic work; a genuine constraint on any commercial path. This is written into `requirements.txt`, the deployment runbook and the licence note.
- **MediaPipe was not simply preferred and ignored.** The `mp.solutions.pose` API the earlier prototypes used has been removed from current MediaPipe releases; the replacement is the Tasks `PoseLandmarker`. The migration was not a like-for-like swap either way.

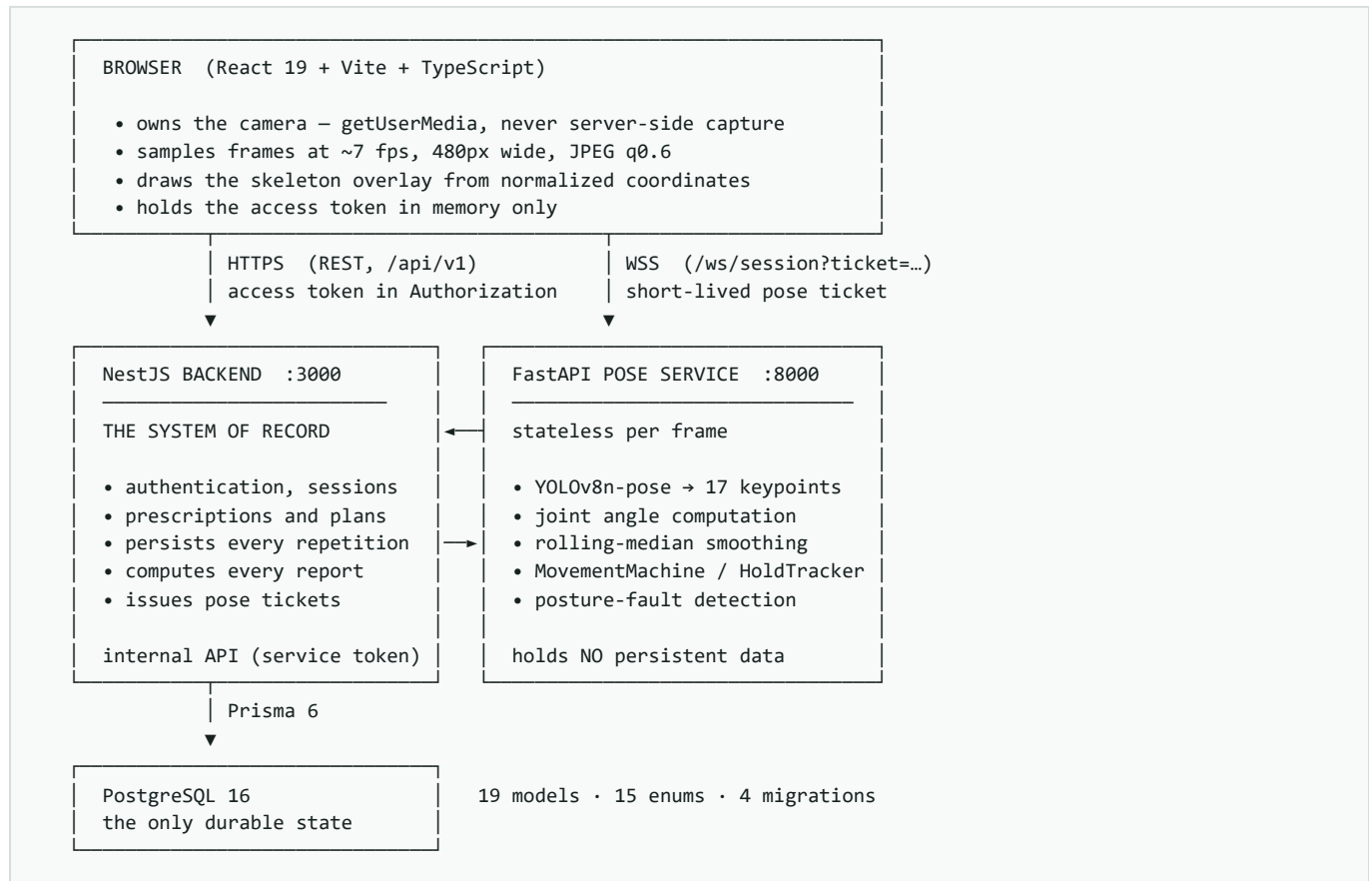
What is genuinely new in this system

- The entire NestJS backend and its 19-model relational schema
- The authoritative trust model and the internal service API
- Database-resident, versioned rule configuration

- The guarded repetition state machine (hysteresis, dwell, cooldown, visibility gating)
- Rolling-median angle smoothing
- The hold-goal path for non-repetition exercises
- The complete authentication and authorisation system
- The entire React frontend and design system
- 305 automated tests
- The containerised deployment and its production hardening

III. System architecture

Three processes, one database, and a deliberate rule about which of them is allowed to be believed.



Why the computer vision is a separate service

This is a question examiners ask, and “because the brief said so” is a weak answer. The defensible reasons:

- Language fit.** The pose-estimation ecosystem — PyTorch, Ultralytics, OpenCV, NumPy — is Python. Reimplementing it in Node would mean either native bindings or an inferior model.
- Failure isolation.** Pose inference is the component that can exhaust CPU or memory. Isolating it means a struggling CV service degrades live sessions but cannot take down authentication, dashboards or the database.
- Independent scaling.** Inference is CPU-bound and expensive; CRUD is cheap. They have different resource profiles and, in the production compose file, different resource limits (2 GB and 2 vCPU for the pose service, 512 MB for the backend).
- Enforceable trust boundary.** Because it is a separate process reached over an authenticated internal API, the rule “the pose service may propose, only the backend may persist” is structural rather than a convention someone could forget.

Why the camera stays in the browser

The inherited FastAPI prototype called `cv2.VideoCapture(0) server-side` — meaning it could only ever analyse a person standing in front of the server. That is fine for a laptop demo and fatal for a remote-monitoring product. Capture is therefore a browser responsibility; the service receives only individual encoded frames over the WebSocket.

The three trust zones

ZONE	TRUSTED FOR	RATIONALE
Browser	Nothing but intent	Fully under user control. It may say “start a session for assignment X” and send frames. It may not say how many repetitions were performed or what the score was.
Pose service	Proposals	Authenticated with a shared service token, but it still only <i>proposes</i> repetitions. The backend validates and decides.
Backend	Everything	Sole writer to the database. Every figure in a report is derived from rows it persisted and validated itself.

IV. Technology choices and their justification

For each choice: what it is, and the one sentence that defends it under questioning.

LAYER	CHOICE	DEFENCE
API framework	NestJS 11	Opinionated modular structure with dependency injection, so a system with 10 controllers and 16 services stays navigable. Decorator-driven validation and OpenAPI generation come from the same metadata.
Database	PostgreSQL 16	The domain is inherently relational and the integrity constraints are clinical, not cosmetic: a session must belong to an assignment, which must belong to a plan, which must belong to a real therapist–patient link. Referential integrity enforces that; a document store would leave it to application code.
ORM	Prisma 6	Types are generated from the schema, so a query result and the TypeScript type describing it cannot drift. Migrations are versioned files applied deterministically by <code>migrate deploy</code> .
CV service	FastAPI + uvicorn	Native async, first-class WebSocket support, and Pydantic models that make the wire protocol a validated schema rather than loose dictionaries.
Pose model	YOLOv8n-pose	The <i>nano</i> variant: 6.8 MB of weights, roughly 140 ms per frame on CPU. Real-time feedback on commodity hardware matters more than the last few points of keypoint precision.
Frontend	React 19 + Vite 6	Vite's dev server is near-instant, and the production build is a static bundle a CDN can serve — which is what makes the Vercel half of the deployment possible at all.
Server state	TanStack Query	Caching, refetching and invalidation for server data, which is genuinely different from client state and badly served by putting it in a store.
Client state	Zustand	Small, unopinionated, no provider ceremony. Used for auth and theme only — the two things that really are global.
Forms	react-hook-form + Zod	Uncontrolled inputs avoid a re-render per keystroke, and the same Zod schema shape validates on both sides of the wire.
Styling	Tailwind CSS v4	The v4 <code>@theme</code> block makes the design system a set of CSS custom properties, which is precisely what made a complete dark theme affordable — see Part XII.
Password hashing	Argon2id	OWASP's current recommendation, at their recommended parameters (19 MiB, 2 iterations, parallelism 1). The <code>@node-rs</code> build ships prebuilt binaries, so installation works on a stock Windows machine with no C++ toolchain — which matters on demo day.

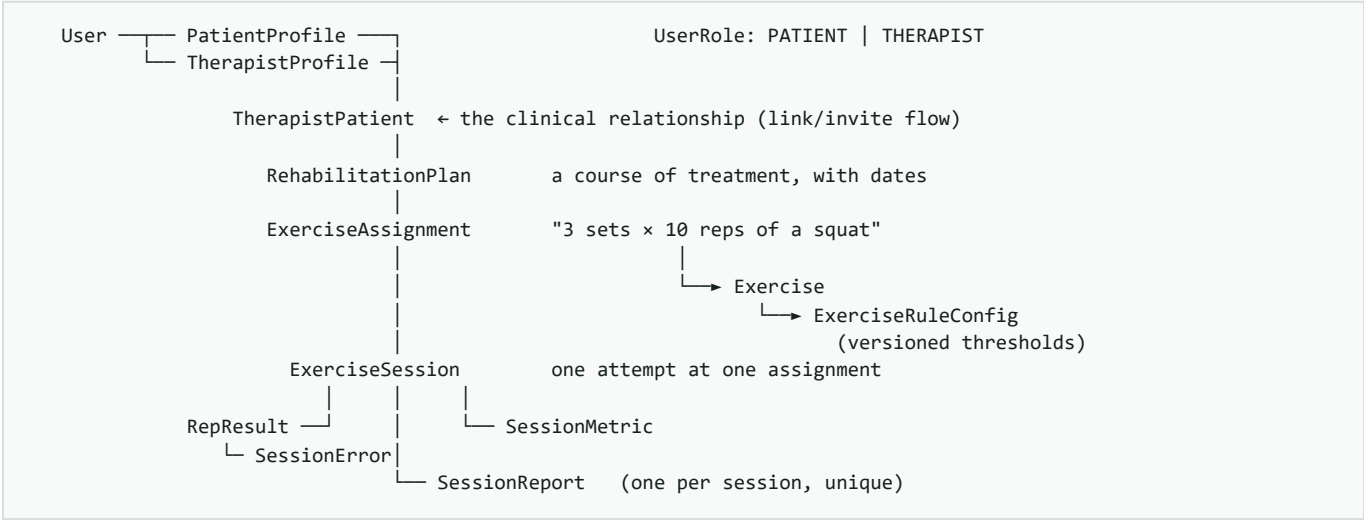
Repository layout

```
physio-ai-platform/
├── apps/
│   ├── backend/      NestJS · Prisma · PostgreSQL    ~9,200 LOC
│   └── frontend/     React · Vite · Tailwind v4      ~9,800 LOC
├── services/
│   └── pose-service/  FastAPI · YOLOv8 (not a workspace)~4,600 LOC
├── docs/              deployment runbook, migration notes, build status
├── legacy/ FYP/ backend/ frontend/  superseded prototypes, kept for provenance
├── docker-compose.yml  local development stack
├── docker-compose.prod.yml  EC2 production stack (standalone)
└── Caddyfile           TLS termination and routing
```

The Python service is deliberately *not* an npm workspace: it has its own dependency manager, its own test runner and its own release cadence. Forcing it into the JavaScript toolchain would buy nothing.

V. The data model

19 models and 15 enums across 697 lines of Prisma schema. Six of them carry the system.



The six that matter

MODEL	ROLE AND THE DETAIL WORTH KNOWING
Exercise	The catalogue entry: name, slug, difficulty, camera view, goal type. Static reference data.
ExerciseRuleConfig	The clinical brain. Required joints, angle definitions, state thresholds and posture rules, all as JSON. Versioned and flagged <code>isActive</code> . This is what makes the system tunable without a deployment.
ExerciseAssignment	The prescription: which exercise, how many sets and repetitions, within which plan. Has a lifecycle (<code>AssignmentStatus</code>) so a therapist can pause or remove it.
ExerciseSession	One attempt. Captures the exercise, targets <i>and the rule-config version</i> at creation time, so the judging criteria are frozen for that session. Moves <code>CREATED</code> → <code>ACTIVE</code> → <code>FINALIZING</code> → <code>COMPLETED</code> (or <code>CANCELLED</code> / <code>ABANDONED</code>).
RepResult	One row per <i>counted</i> repetition: number, score, correctness flag, tracking confidence, angle summary. Written only by the backend, via the internal API.
SessionReport	The finalized summary. <code>sessionId</code> is unique, which is what makes repeated completion idempotent rather than a duplicate-key error.

Three schema decisions worth defending

- **Profiles split from `User`.** Authentication data (email, password hash, status) is shared; clinical data is not. A patient has a date of birth and a condition; a therapist has a licence number and a specialisation. One nullable-everything table would model neither honestly.
- **Partial unique indexes.** A dedicated migration (`20260828040000_partial_unique_indexes`) adds uniqueness that applies only to rows in a particular state — for example, at most one *active* link between a given therapist and patient, while historical inactive links remain.
- **Errors as rows, not a text blob.** `SessionError` stores a structured fault code per occurrence, which is what allows the report to aggregate “insufficient depth, 7 of 12 repetitions” rather than printing a paragraph.

MIGRATION DISCIPLINE

Four migrations, applied automatically by the backend's Docker entrypoint on every start via `prisma migrate deploy`. Not `migrate dev` — that is an interactive development command that can prompt and can reset. The production path is deterministic and non-interactive by construction.

VI. The computer-vision pipeline

Six stages turn a JPEG into a judgement. Being able to walk an examiner down this list, in order, is the core technical demonstration of the project.

1. CAPTURE	browser · ~7 fps · 480px wide · JPEG quality 0.6 ↓ base64 over WebSocket
2. DECODE	frame_decoder.py → NumPy array (bounded size, guarded) ↓
3. DETECT	YOLOv8n-pose → N people × 17 COCO keypoints (x, y, confidence) ↓ select_primary_pose() picks deliberately, warns on MULTIPLE_PEOPLE
4. MEASURE	geometry/angles.py → named joint angles in image-pixel space ↓
5. SMOOTH	rolling MEDIAN over 5 frames (≈0.5 s), per angle ↓
6. JUDGE	MovementMachine → repetition events posture rules → faults HoldTracker → seconds accumulated → per-frame score ↓
PERSIST	pose service POSTs to the backend's internal API; the BACKEND decides whether the repetition is recorded.

Stage 3 — detection

YOLOv8n-pose returns the 17-point COCO keypoint set (nose, eyes, ears, shoulders, elbows, wrists, hips, knees, ankles) as 2-D pixel coordinates with a per-keypoint confidence. Landmarks are addressed *by name* through a registry, not by raw index — `LEFT_KNEE`, not `keypoints[13]` — so a model change or index shift is a one-line map update rather than a hunt through the analysis code.

Three deliberate departures from the inherited prototype:

- **No frame is drawn on.** The prototype rendered an annotated JPEG server-side and shipped it back. The browser already has the video; it draws the skeleton itself from normalized coordinates. Encoding a second image server-side would be wasted work on the hot path.
- **All detected people are returned.** The prototype took `keypoints.xy[0]` — silently analysing whichever person YOLO happened to list first. `select_primary_pose()` chooses deliberately and the interface warns the patient when it had to choose.
- **The model is loaded once and shared.** YOLOv8 inference is stateless per call, so there is nothing to leak between sessions and no per-session model instance. Concurrency is capped by an explicit slot count instead.

Stage 4 — angle measurement

Angles are computed in **image-pixel space**, which is what YOLOv8 returns and what the inherited thresholds were tuned against. The geometry module provides `angle_between` (three points), `angle_from_vertical`, `midpoint`, `horizontal_distance`, `vertical_distance` and `to_reference_scale`.

Composite angles are evaluated in a second pass and combine other *named* angles rather than landmarks directly. The four combinators are worth memorising, because the reasoning behind one of them is a strong answer:

COMBINATOR	MEANING AND WHEN IT IS CORRECT
average	Both sides move together — a squat, where the knees flex as a pair.
min	The most-flexed side drives the movement. Required for a single-limb exercise whose angle decreases, such as a bicep curl.
max	The most-extended side drives it — the equivalent for an increasing angle, such as a single-arm raise.
ratio	One measurement divided by another, for rules comparing two distances rather than testing one against a constant. The squat's knee-alignment check is exactly this.

THE SINGLE-LIMB AVERAGING BUG — A VERY GOOD THING TO BE ASKED ABOUT

Averaging is wrong for a single-arm bicep curl, and the failure is silent. One arm curled to 50° beside one resting at 170° averages to 110°. A 55° peak threshold is never crossed, so *no repetition can ever complete* — the patient exercises correctly and the counter stays at zero. That is why `min` exists as a combinator, and why `test_single_limb.py` is a dedicated test file.

Stage 5 — smoothing

Raw pose estimates jitter by a few degrees even when the patient is still. A **rolling median** over a five-frame window is applied per angle — median rather than mean, because the median rejects an occasional wildly-wrong landmark outright where a mean would be dragged toward it. The window is deliberately small (about 0.5 s at 10 fps) so the added latency stays well under human reaction time.

Stage 6 — fault detection

Nineteen structured fault codes across four categories, each carrying a severity (`INFO` , `WARNING` , `CRITICAL`) and a score penalty:

CATEGORY	CODES
Tracking	NO_PERSON_DETECTED · PARTIAL_BODY_VISIBLE · LOW_CONFIDENCE · MULTIPLE_PEOPLE · FRAME_DECODE_ERROR
Squat	INSUFFICIENT_DEPTH · EXCESSIVE_DEPTH · TRUNK_LEAN · ASYMMETRIC_LOAD · KNEE_ALIGNMENT
Upper limb	ELBOW_DRIFT · INCOMPLETE_ROM · SHOULDER_HITCH · TRUNK_COMPENSATION
Posture & general	KNEE_FLEXED · SHOULDER_ALIGNMENT · HIP_ALIGNMENT · SPINE_ALIGNMENT · MOVEMENT_TOO_FAST

Tracking codes are separated from form codes for an important reason: a tracking failure means *the system could not see properly*, which is not the same as the patient performing badly and must not be penalised as though it were.

VII. Repetition counting — the state machine

This module is the strongest single piece of engineering in the project. If you prepare one deep answer, prepare this one.

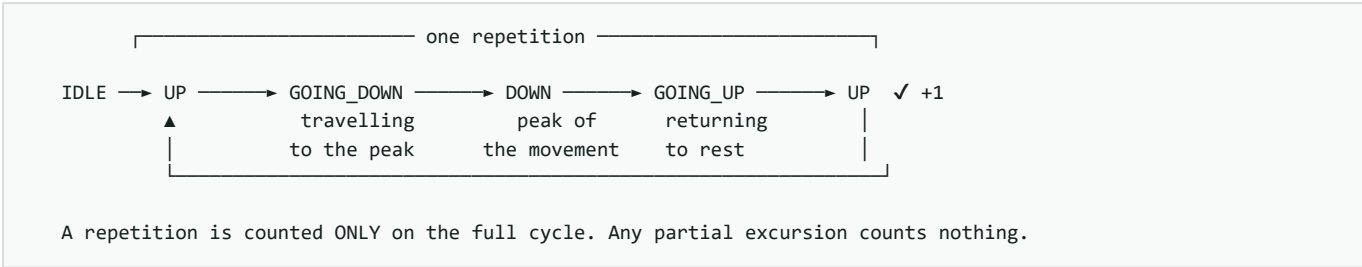
The problem with the obvious approach

A repetition is not “a correct frame was seen”. Counting per frame, or on a single threshold crossing, produces two failure modes that *both existed in the inherited prototypes*:

- **Jitter double-counting.** Pose estimates wobble a few degrees. An angle hovering near the threshold crosses it repeatedly, adding several repetitions for one movement.
- **Half repetitions.** Going down and stopping counts as a repetition if you only watch one boundary.

The prototype's `_update_rep_counter` flipped on a bare `prev_state → new_state` comparison with no hysteresis, no dwell requirement and no cooldown. Its *thresholds* were sound and were preserved exactly; its transition logic was replaced wholesale.

The cycle



A NAMING TRAP TO PRE-EMPT

DOWN means *peak of the movement*, not “physically lower”. For a squat it reads literally. For a bicep curl, DOWN is the *curled* position; for a shoulder raise it is the *raised* position. The rule config's `direction` field records which way the angle actually travels. Examiners notice this, and having the answer ready is worth a lot.

The four guards

GUARD	WHAT IT PREVENTS
Hysteresis	The angle must travel past the threshold by an additional margin before the state can flip back. This is the direct fix for jitter double-counting: a wobble of two degrees around the boundary no longer toggles anything.
Minimum dwell	A state must persist for N consecutive frames before it is accepted, so one stray landmark cannot advance the cycle.
Cooldown	A short refractory period after each counted repetition — a second guard against a single movement registering twice.
Visibility gating	The machine is <i>frozen entirely</i> while the required landmarks are not reliably visible. Walking out of frame can therefore never advance the cycle; it pauses it.

The hold path

`HoldTracker` is the counterpart for `static-posture`. It counts nothing and accumulates *seconds held in correct alignment*. Its completion criterion is seconds held against seconds prescribed, rather than repetitions performed against repetitions prescribed, and it produces a cumulative snapshot rather than a series of per-repetition rows. That difference propagates all the way to `finalizeHoldSession()` in the report service, which explicitly writes zero repetitions rather than leaving stale non-zero values behind.

Resumption

The analyzer is constructed with `initial_rep_count` and `initial_held_seconds`, seeded from the database. If a WebSocket drops mid-session and the browser reconnects, the count resumes from what was *persisted* rather than restarting at zero — another consequence of the backend being authoritative.

VIII. Data-driven rules and clinical versioning

The design goal: adding an exercise should be a database row, not a Python class.

Every one of the five exercises uses `BaseExerciseAnalyzer` *unchanged*. They differ only in their `ExerciseRuleConfig` row — required joints, angle definitions, state thresholds, posture rules. The registry exists purely so that an exercise which genuinely needs bespoke Python can be registered without changing any call site; the overrides map is currently empty, which is the intended state.

What moved out of code and into the database

The prototype hard-coded `if knee_angle > 160` inside `SquatAnalyzer`. Here that arrives as JSON. Two things follow:

- **A physiotherapist can tune the system without a developer.** Changing the depth threshold for a squat is an update to a row, not a code change, a code review, a build and a deployment.
- **A session records which rule *version* judged it.** This is the part that matters clinically.

THE VERSIONING ARGUMENT — WORTH REHEARSING

Suppose a therapist decides in March that the squat depth threshold was too lenient and tightens it. Without versioning, every report generated in January silently re-interprets against criteria that did not exist when the patient performed the exercise, and a patient's apparent progress changes retroactively because the ruler moved.

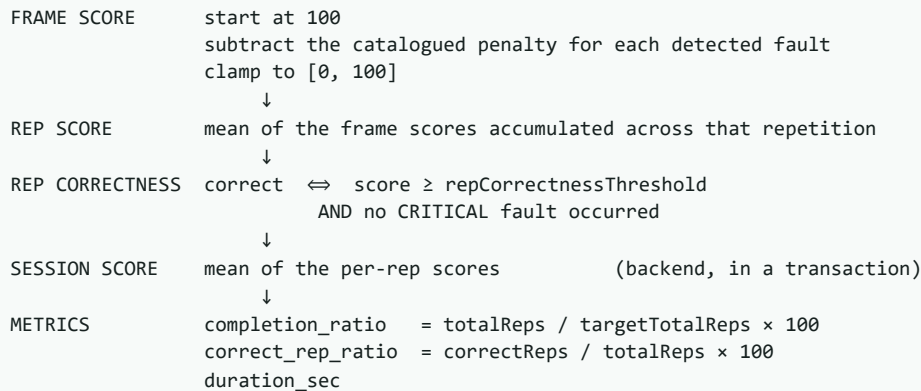
Instead, a changed rule config is written as a **new version** with the previous one retired, and each `ExerciseSession` stores the version that judged it. Historical reports keep the thresholds that actually judged them. The seeding tool is idempotent and matches by slug, so re-running it is safe.

Structure of a rule config

```
ExerciseRuleConfig
├── requiredJoints    ["LEFT_KNEE", "RIGHT_KNEE", "LEFT_HIP", ...]
│                   ← drives visibility gating; missing ⇒ machine frozen
├── angleDefinitions  named angles: three-point, from-vertical,
│                   or composite (average | min | max | ratio)
├── stateThresholds   peak / rest boundaries, hysteresis margin,
│                   minimum dwell frames, cooldown, direction
├── postureRules      per-frame form checks → fault codes
└── repCorrectnessThreshold  the score at or above which a rep is "correct"
```

IX. The scoring model

Deliberately simple, bounded, explainable and testable — not an opaque “AI accuracy” figure.



Why it is built this way

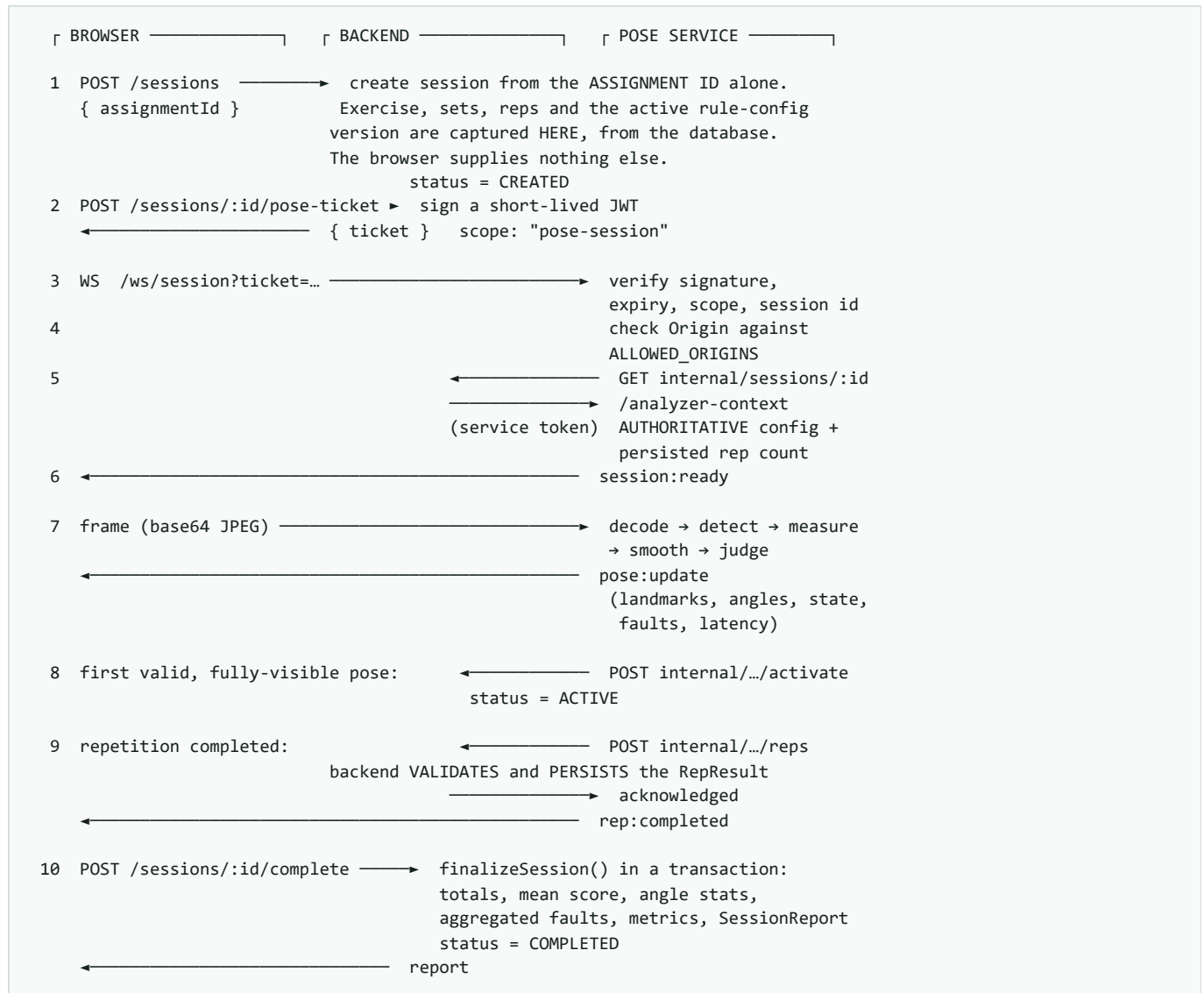
- **Every number is traceable.** A patient asking “why did I score 78?” gets an answer: these repetitions, these faults, these penalties. A neural scoring model could not answer that, and in a clinical context an unexplainable score is close to useless.
- **Correctness needs both conditions.** Score alone is insufficient. A repetition could average above the threshold while containing one **CRITICAL** fault — a knee-alignment failure, say — and calling that “correct” would be exactly wrong, because a single instance of that fault is the one that causes injury.
- **Tracking faults are separated from form faults.** A patient must not be marked down for standing in poor light.
- **It is computed in a transaction.** `finalizeSession()` runs the whole computation inside `prisma.$transaction`, so a crash cannot leave a session marked **COMPLETED** with no report, or a report with stale totals. The report is `upsert` ed on a unique `sessionId`, which makes repeated completion idempotent.

ANTICIPATE THIS CHALLENGE: “WHERE IS THE AI?”

An examiner may press on the fact that the scoring itself is rule-based arithmetic. The honest and strong answer: *the machine learning is in the perception layer — a convolutional neural network (YOLOv8) performing multi-person 2-D pose estimation, which is the genuinely hard and genuinely learned part. The clinical judgement layer on top of it is deliberately rule-based, because a physiotherapist must be able to inspect, tune and justify the thresholds that judge their patient. Making the scoring a black box would have made the system less useful, not more advanced.*

X. The live session, end to end

Ten steps. This is the demo narrative, and the sequence an examiner is most likely to ask you to trace.



Four details in that flow worth knowing precisely

- Step 1 — the browser supplies only an assignment id.** It cannot inflate its own targets, choose a different exercise, or select a more lenient rule version. Everything the pose service will later be told is read from the database at session creation.
- Step 5 — the pose service asks the backend, not the browser, for its configuration.** The word in the code is “authoritative”. The browser is never in that path.
- Step 9 — the pose service proposes; the backend disposes.** A repetition is not counted because the pose service says so. It is counted because the backend accepted the proposal and wrote the row.
- Ticket expiry is not re-checked per frame.** The ticket authorises *establishing* the connection. Terminating a legitimate three-minute exercise because a two-minute ticket lapsed would be hostile to the patient. A separate absolute cap (`POSE_MAX_SESSION_MINUTES` , default 120) prevents a forgotten socket living forever, and a scheduled sweep closes stale sessions.

Failure handling on the socket

Application-defined WebSocket close codes in the 4000–4999 range distinguish causes rather than collapsing them into a generic failure: `4401` invalid ticket, `4403` forbidden origin, plus structured error messages for

`SESSION_REJECTED`, `UNSUPPORTED_EXERCISE` and `AT_CAPACITY`. Each error carries a `recoverable` flag so the interface knows whether retrying is worth offering.

XI. The trust model and security design

The recurring theme: never trust the client, and make that structural rather than a matter of discipline.

Authentication

MECHANISM	DESIGN AND RATIONALE
Password hashing	Argon2id at OWASP's recommended parameters. <code>verify()</code> returns <i>false</i> rather than throwing on a malformed stored hash, so a corrupt row cannot turn a login attempt into a 500 that leaks internals.
Access token	Short-lived JWT (15 minutes by default), held in memory only in the browser. Not in <code>localStorage</code> , which any injected script can read.
Refresh token	Opaque random bytes, not a JWT. 48 bytes of <code>randomBytes</code> , delivered in an <code>HttpOnly</code> cookie scoped to <code>/api/v1/auth</code> . Only its SHA-256 hash is stored.
Email and reset tokens	Same pattern — stored as SHA-256 hashes because they travel in URLs that end up in mail logs and browser history. Reset tokens are deliberately shorter-lived than verification tokens, because that one can change a credential.
Pose ticket	A separate JWT with <code>scope: 'pose-session'</code> , signed with a <i>different secret</i> . An application access token can never be replayed as a pose ticket, and a pose ticket can never authenticate against the API.

WHY THE REFRESH TOKEN IS OPAQUE — A VERY LIKELY QUESTION

A JWT is self-validating: the server checks the signature and accepts it. That means a *stolen* JWT refresh token stays valid until it expires, even after the user logs out — there is nothing to revoke. An opaque token must be looked up in the `refresh_tokens` table, so revocation is immediate and reuse is detectable. Storing only the SHA-256 hash means a database leak yields nothing replayable.

Account-enumeration resistance

Every endpoint that acts on an email address supplied by an anonymous caller — forgot-password, resend-verification — returns a single constant `NEUTRAL_EMAIL_RESPONSE`, worded identically whether or not the address exists. Anything else turns those endpoints into an oracle that tells an attacker precisely which addresses are registered.

The resend-verification control on the login page is a deliberate exception that proves the rule: it appears only *after* the password has already been accepted, so showing it reveals nothing an attacker did not already know.

Authorisation

A dedicated `AccessControlService` centralises the ownership questions — may this therapist see this patient, does this assignment belong to this patient — rather than scattering the checks across controllers where one omission is invisible. It has its own test suite (`access-control.service.spec.ts`). Relationships are consent-based: a therapist and patient are linked through an invite flow with a time-limited code (`INVITE_CODE_TTL_MINUTES`, default 60).

Transport and infrastructure

- **Helmet** for security headers
- **Throttler** rate limiting (120 requests / 60 s by default)
- **Zod environment validation at boot** — the backend refuses to start on invalid configuration and names the missing variable
- **SMTP required in production**, deliberately: an unsendable verification email locks every new account out of itself permanently
- **PostgreSQL bound to 127.0.0.1** in development and *not published at all* in production
- `/analyze` **is not mounted** when `POSE_ENVIRONMENT=production`, and Caddy does not

route to it either — two independent locks

- **Container runs as UID 10001**, not root

- **Origin checked on the WebSocket handshake** by the service itself, because CORS middleware does not cover upgrades

THE /ANALYZE DOUBLE LOCK — A GOOD THING TO VOLUNTEER

`/analyze` is a debug route that runs full pose inference on any posted image *with no authentication*. Left publicly reachable it hands an anonymous caller the instance's CPU. It is locked twice — the application refuses to mount it in production, *and* the reverse proxy routes only `/ws/session*` to the service at all. Either lock alone would be a single configuration mistake away from being the only one.

XII. Frontend architecture and the design system

The visual concept: the goniometer — the hinged, degree-marked protractor a physiotherapist actually holds against a joint.

Structure

Route-based, with role-guarded segments. Patients reach `/patient/*` (dashboard, exercises, sessions, reports, progress, therapist, notifications) and therapists reach `/therapist/*` (dashboard, patients, patient detail, per-patient progress, session reports, exercise library, notifications). Authentication pages sit outside both. The live session is a full-bleed route of its own, outside the standard chrome, because a camera view competing with a sidebar is a worse experience.

The design system

Everything lives in one file, `apps/frontend/src/index.css`, as Tailwind v4 `@theme` tokens:

- **Colour ramps.** A clinical teal `brand` ramp (with the 400 step reserved for “live”), an `ink` neutral ramp pulled slightly toward teal-green, three semantic ramps (`good`, `caution`, `problem`), and a muted indigo `accent` reserved for a second data series only.
- **Instrument components.** A `Goniometer` dial, a `TallyTrack`, a `ProgressRing`, `SignalBars`, `Measure` and `RangeBar` — readouts modelled on clinical instruments rather than generic dashboard widgets.
- **Motion.** Scroll-triggered reveals, mount staggers, pointer-positioned ripples, page transitions and eased dial values, built into the `Card` and `StatStrip` primitives so that animation arrives everywhere without wrapping fifteen files by hand.

Dark mode by ramp inversion

A DESIGN DECISION WORTH EXPLAINING

The naive approach is a `dark`: variant on every colour utility — roughly two hundred classes to write and maintain, each an opportunity to miss one. Instead the `[data-theme='dark']` block **redefines the ramps themselves**: `ink` inverts (50 ↔ 900), `brand` and semantic 50-tints become dark and their 700-text steps become light, while the 300 and 500 steps stay put because they are used where contrast works in both directions. Roughly two hundred existing colour classes therefore carried over untouched, and a complete second theme cost about 2.4 KB of CSS.

Three surfaces must stay dark in *both* themes — the camera console, the “today” hero panel and the authentication panel — so they use a separate, non-inverting `stage` palette.

A blocking inline script in `index.html` sets `data-theme` before first paint, from `localStorage` or `prefers-color-scheme`, wrapped in try/catch. Without it the page flashes light before the React bundle loads. The store has three states — light, dark, and *not chosen, follow the system* — which is genuinely different from defaulting to light.

Accessibility and motion

- `prefers-reduced-motion` zeroes **both** durations and delays. Zeroing only durations leaves a staggered element invisible for the length of its delay — worse than no animation at all.
- Live regions are updated via the Web Animations API rather than by remounting on a changed key, because remounting an ARIA live region makes screen readers announce nothing.
- `SignalBars` is `aria-hidden`; the redundant encoding is the bar *count*, so the information does not depend on colour.

A REAL ENGINEERING STORY FROM THIS PROJECT — THE SKELETON OVERLAY

An early revision eased every landmark toward its newest position each animation frame, to make the skeleton look smoother. Easing *toward* a target means never quite arriving at one: while the patient was actually moving, the skeleton sat permanently behind them and read as lag. It was reverted to drawing each result exactly, and the canvas glow effect was dropped as well — a per-frame GPU cost on a machine already running inference. The file now carries a load-bearing comment warning against reintroducing interpolation. This is a good answer to “tell us about something that did not work”.

XIII. Testing strategy

305 tests, all passing	74 backend (Jest)	131 pose service (pytest)	100 frontend (Vitest)
----------------------------------	-----------------------------	-------------------------------------	---------------------------------

What is tested, and why those things

SUITE	FOCUS
pose: test_movement.py	The state machine — hysteresis, dwell, cooldown, visibility gating, and that partial excursions count nothing. The highest-value tests in the project.
pose: test_single_limb.py	The averaging bug specifically: that a one-armed curl still completes repetitions.
pose: test_geometry.py	Angle mathematics against known values.
pose: test_hold.py	Seconds accumulate only while alignment holds.
pose: test_session_autoclose.py	Abandoned sessions are closed rather than living forever.
pose: test_analyzer.py / test_yolo_detector.py	End-to-end analysis and detector behaviour, including multi-person selection.
backend: reports.service.spec.ts	Score computation, both goal types, idempotent finalization.
backend: access-control.service.spec.ts	Ownership rules — that a therapist cannot read a patient they are not linked to.
backend: password / env.validation / email.templates	Hashing behaviour, boot-time configuration refusal, email rendering.
backend: app.e2e-spec.ts	Full HTTP flows against an embedded PostgreSQL instance.
frontend: session / hold-session / session-auto-finish	The live session interface across both goal types, including automatic completion.
frontend: assignment-actions / patient-archive / routing / camera / password	Therapist workflows, route guards, camera permission states, password rules.

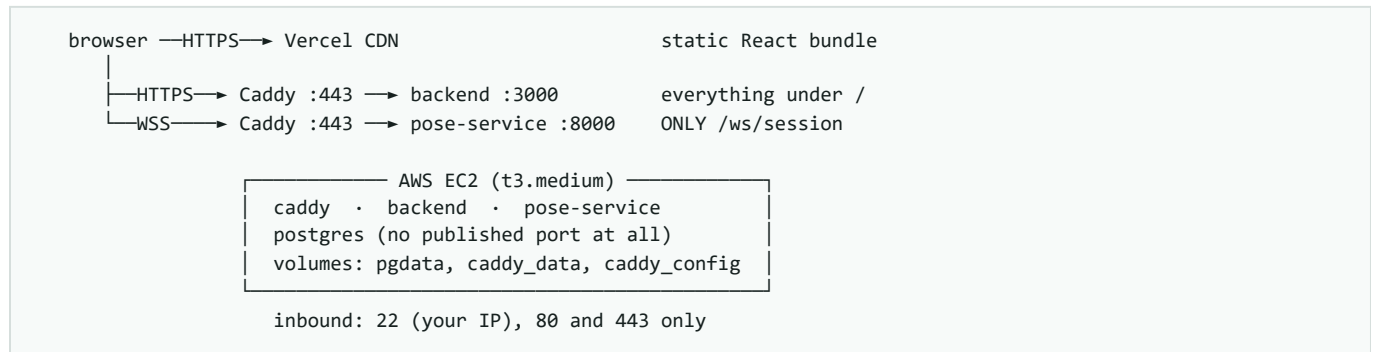
THE TESTING PHILOSOPHY, IN ONE SENTENCE

The tests concentrate on the places where *being subtly wrong is worse than crashing* — a repetition counter that overcounts, a score that is silently inflated, an access check that lets the wrong therapist read a patient's data. A crash is visible; a plausible wrong number is not.

An embedded PostgreSQL (`embedded-postgres`) is used for end-to-end tests, so the suite runs on a machine with no database installed — the same reasoning that drove the `@node-rs/argon2` choice.

XIV. Deployment architecture

A split deployment: static frontend on a CDN, stateful services on one instance.



Why this split

The frontend is a static bundle; nothing about it needs a server, so a CDN serves it better and cheaper than an instance would. The backend holds the database and the pose service holds a loaded model in memory — both are stateful in ways a serverless platform is not shaped for.

THE CONSEQUENCE THAT DRIVES EVERYTHING ELSE

The two halves are on *different origins*, so the browser holds an HTTPS page from Vercel and calls an HTTPS API on EC2. Three things follow, and they are a chain:

1. TLS on the EC2 side is not optional — a page served over HTTPS may not call `http://` or open `ws://`, and the browser blocks it with no fallback. Hence Caddy and Let's Encrypt.
2. The refresh cookie is cross-site, so it needs `SameSite=None; Secure`, which requires `COOKIE_SECURE=true`.
3. That in turn requires `TRUST_PROXY=true`, because Caddy terminates TLS and Express would otherwise believe the connection was plain HTTP and quietly decline to set a secure cookie — sign-in appears to succeed and the session is gone on reload.

Production hardening applied

MEASURE	REASON
Multi-stage Docker builds with <code>.dockerignore</code>	Smaller images; and excluding <code>*.tsbuildinfo</code> fixes a real failure where a stale incremental-build file made <code>tsc</code> a silent no-op after the output directory was wiped.
CPU-only torch wheels, installed first	The GPU build is several gigabytes and useless on a CPU instance. Installing it as its own layer also keeps the expensive step cached.
Model weights committed, verified by checksum	The download step becomes a 6.8 MB checksum verification rather than a network dependency that can fail mid-build.
Migrations in an entrypoint, using <code>exec</code>	<code>exec "\$@"</code> replaces the shell so the application is PID 1 — without it SIGTERM never reaches NestJS's shutdown hooks and the container is killed rather than stopped.
Seed script excluded from the production image	<code>prisma/seed.ts</code> deletes every session, report and assignment and creates accounts on a shared published password. There is deliberately no <code>dist/tools/seed.js</code> .
Non-root container user (UID 10001)	Standard least-privilege.
<code>HEALTHCHECK</code> on every service, 90 s start period for the pose service	Loading torch and the model on a burstable instance is genuinely slow; a shorter period would mark a healthy service as failed.
Log rotation (3 × 10 MB) and memory limits	Neither logs nor a leaking process can fill the disk or the instance.
<code>POSE_MAX_CONCURRENT_SESSIONS=3</code>	One YOLOv8n frame costs roughly 140 ms of CPU, so a 2-vCPU instance sustains about 7 fps <i>in total</i> . Three concurrent patients at 7 fps already oversubscribes it; more would only queue frames that backpressure then drops.
Standalone production compose file	Not an overlay. A <code>-f base -f override</code> pair cannot <i>remove</i> a service, and running the wrong combination on a server is exactly the mistake worth designing out.

Sizing. `t3.medium` (2 vCPU / 4 GB) is the floor: the pose service sits at roughly 1.5–2 GB resident during a session, and a `t3.small` would be OOM-killed mid-exercise. Two gigabytes of swap turns the peak of first model load from a hard kill into a slow first request.

XV. Limitations and future work

Examiners reward candidates who name their own limitations before being shown them. Volunteer these.

Honest limitations

LIMITATION	THE CONSIDERED RESPONSE
No clinical validation	The thresholds come from the inherited prototype and general biomechanics, not a trial against goniometer ground truth. Validating them would require a study with a physiotherapist and instrumented measurement — out of scope for a final-year project, and the honest answer is to say so rather than overclaim.
2-D pose estimation only	Angles are measured in the image plane, so movement toward or away from the camera is foreshortened. This is mitigated by prescribing a camera view per exercise (the <code>CameraView</code> enum), not solved. Depth cameras or a monocular 3-D lifting model would solve it properly.
Single-camera, single-person	Multiple people are detected and one is selected deliberately with a warning, but occlusion by furniture or a second person still degrades tracking.
Throughput ceiling	Roughly three concurrent sessions on a t3.medium. Horizontal scaling is possible — the pose service is stateless per frame — but is not implemented.
AGPL-3.0 dependency	Ultralytics' licence is copyleft over network use. Fine academically; a commercial path would need either a commercial licence from Ultralytics or a differently-licensed model.
No offline mode	A dropped connection pauses the session. Resumption from the persisted count exists; local buffering does not.

Future work, in order of value

1. **Clinical validation study** — compare measured angles against goniometer ground truth and tune the rule configs from real data. Everything else is less valuable than this.
2. **Therapist-facing rule editing in the interface.** The data model and versioning already support it; only the editing surface is missing. This is the shortest path from “configurable in principle” to “configurable in practice”.
3. **Progress prediction.** A `ProgressEstimate` model already exists in the schema for trend estimation — a genuine machine-learning extension on top of the accumulated longitudinal data.
4. **3-D pose or depth sensing** to remove the image-plane limitation.
5. **Native mobile application** — better camera control and the ability to prop a phone up where a laptop will not go.
6. **Horizontal scaling of the pose service** behind a load balancer, which the stateless design already permits.

XVI. Examiner Q&A bank

Twenty-six questions, grouped. Each answer is grounded in code that exists in this repository — nothing here is invented for the viva.

ARCHITECTURE AND DESIGN

Q Why did you split the computer vision into a separate service instead of doing it all in one backend?

Four reasons. The pose ecosystem — PyTorch, Ultralytics, OpenCV — is Python, so reimplementing it in Node would mean native bindings or a worse model. Inference is the component that can exhaust CPU and memory, so isolating it means a struggling CV service cannot take down authentication or the database. The two have completely different resource profiles and get different container limits. And because it is a separate process reached over an authenticated internal API, the rule that only the backend may persist data is structural rather than a convention someone could forget.

Q Why is the camera in the browser rather than on the server?

The inherited prototype called `cv2.VideoCapture(0)` server-side, which means it could only ever analyse someone standing in front of the server. That is fine for a laptop demo and fatal for remote monitoring, which is the entire point of the product. Capture belongs to the browser; the service receives individual encoded frames.

Q Why PostgreSQL rather than MongoDB? One of your earlier prototypes used MongoDB.

The domain is inherently relational and the constraints are clinical rather than cosmetic. A session belongs to an assignment, which belongs to a plan, which belongs to a real therapist–patient link. A session whose assignment does not exist is not an edge case to handle gracefully — it is corrupt clinical data. Referential integrity enforces that at the database; a document store would leave it to application code, where one missed check is invisible. We also make real use of relational features: partial unique indexes, transactional report generation, and aggregate queries across sessions for progress trends.

Q What is the most significant architectural decision in the project?

That the backend is authoritative over every number. The inherited React prototype posted a literal `accuracy: 80` — a hard-coded constant presented as a measurement. In this system the browser may only say “start a session for this assignment” and send frames; the pose service may only *propose* a repetition; and the backend validates, persists and computes the report from its own rows. Everything else in the design follows from that rule.

THE COMPUTER VISION

Q Walk us through what happens to a single frame.

Six stages. The browser captures at about seven frames per second, 480 pixels wide, JPEG quality 0.6, and sends it over the WebSocket. The service decodes it to a NumPy array. YOLOv8n-pose detects every person in the frame as seventeen COCO keypoints with per-keypoint confidence, and `select_primary_pose` picks the subject deliberately. The geometry module computes the named joint angles for that exercise in image-pixel space. Each angle passes through a five-frame rolling median. Then the state machine decides whether a repetition just completed, and the posture rules produce a per-frame score and any fault codes. If a repetition completed, the service posts it to the backend's internal API — and the backend decides whether it is recorded.

Q Why YOLOv8 and not MediaPipe?

Two parts to this, and I want to be straightforward about the history. The project brief specified no YOLO, but a YOLOv8 dependency was already embedded in the inherited FastAPI prototype, along with the exercise analysis code that is the most valuable prior work in the project. Separately, the `mp.solutions.pose` API those prototypes used has since been removed from MediaPipe; the replacement is the Tasks `PoseLandmarker`, so it would not have been a like-for-like swap in either direction. We kept YOLOv8n deliberately, and we document the consequence — Ultralytics is AGPL-3.0, which is copyleft over network use, so hosting this publicly obliges us to offer the source under the same terms.

Q Why the *nano* variant? Wouldn't a larger model be more accurate?

It would, marginally, at several times the inference cost. The requirement is real-time feedback on commodity hardware: a nano frame costs roughly 140 ms of CPU, which sustains about seven frames per second on two vCPUs. Feedback that arrives after the repetition is finished has no clinical value, so latency dominates the last few points of keypoint precision. The model path is configuration, so a larger variant is a deployment decision rather than a code change.

Q Why a median filter rather than a mean or a Kalman filter?

A median rejects an occasional wildly-wrong landmark outright, where a mean is dragged toward it — and a badly-misplaced keypoint is exactly the failure mode pose estimation has. A Kalman filter would model the motion and could do better, but it needs a motion model per exercise and adds tuning parameters; a five-frame median is about 0.5 s of latency at 10 fps, which is well under human reaction time, and it is trivially testable. We also learned the hard way that *too much* smoothing is worse than none — see the skeleton overlay story.

Q How do you handle more than one person in the frame?

The detector returns all detected people rather than silently taking index zero, which is what the prototype did. `select_primary_pose` chooses the subject deliberately, and a `MULTIPLE_PEOPLE` code is raised so the interface can tell the patient that someone else is in shot. It is a tracking code rather than a form code, so it does not penalise the patient's score — it means the system could not see properly, which is a different thing from performing badly.

REPETITION COUNTING

Q How do you count a repetition?

With a state machine, not a threshold test. The angle must complete the full cycle `UP → GOING_DOWN → DOWN → GOING_UP → UP` before anything is counted, and four guards protect it: hysteresis, so the angle must travel past the threshold by an extra margin before the state can flip back; a minimum dwell of N consecutive frames, so one stray landmark cannot advance the cycle; a cooldown after each counted repetition; and visibility gating, which freezes the machine entirely while the required landmarks are not reliably visible.

Q Why not just count when the angle crosses a threshold?

Because that has two well-known failure modes, and both existed in our own prototypes. Pose estimates wobble a few degrees, so an angle hovering near the threshold crosses it repeatedly and adds several repetitions for one movement — hysteresis fixes that. And if you only watch one boundary, going down and stopping counts as a full repetition — requiring the whole cycle fixes that. The prototype's thresholds were actually fine and we preserved them exactly; it was the transition logic we replaced.

Q Your states are called `UP` and `DOWN`. What does `DOWN` mean for a bicep curl?

The curled position. `DOWN` means *peak of the movement*, not physically lower — for a shoulder raise it is the raised position. Which way the angle actually travels is recorded in the rule config's `direction` field. The names read literally for a squat, which is where they came from, and the docstring flags the trap explicitly.

Q What happens if the patient walks out of frame mid-set?

The machine freezes. Visibility gating means that while the required landmarks for that exercise are not reliably visible, no state transition is accepted at all — so walking out of frame pauses the cycle rather than advancing it, and cannot produce a phantom repetition. When they return, it resumes from the state it was in. If the connection itself drops, the analyzer is rebuilt seeded from the count the backend has *persisted*, so the patient does not restart at zero.

Q You said averaging both limbs caused a bug. Explain it.

For a single-arm bicep curl, averaging the two elbow angles is silently wrong. One arm curled to 50 degrees beside one resting at 170 averages to 110. A 55-degree peak threshold is never crossed, so no repetition can ever complete — the patient performs the exercise correctly and the counter stays at zero, with no error shown. That is why the `min` combinator exists: for a single-limb exercise whose angle decreases, the most-flexed side drives the movement. `max` is the equivalent for an increasing angle. There is a dedicated test file for exactly this case.

SCORING AND CLINICAL LOGIC

Q How is the score calculated?

Each frame starts at 100 and loses a catalogued penalty for every detected fault, clamped to the range 0 to 100. A repetition's score is the mean of its frame scores. The session score is the mean of the repetition scores, computed by the backend inside a transaction. A repetition counts as *correct* only if it scores at or above the exercise's correctness threshold *and* contains no `CRITICAL` fault — both conditions, because a repetition could average well while containing one knee-alignment failure, and that single instance is the one that causes injury.

Q That is just arithmetic. Where is the AI in your AI-powered system?

In the perception layer, which is the genuinely hard part — a convolutional neural network performing multi-person 2-D pose estimation, turning raw pixels into seventeen anatomical landmarks. That is a learned model doing something no rule could do. The clinical judgement layer on top is deliberately rule-based, and that is a design decision rather than a shortfall: a physiotherapist has to be able to inspect, tune and justify the thresholds that judge their patient, and a patient asking “why did I score 78?” deserves an answer. Making the scoring a black box would have made the system less useful clinically, not more advanced.

Q Why are the thresholds in the database instead of in the code?

Two reasons. First, a physiotherapist can retune an exercise without a developer, a code review, a build and a deployment — and they are the person qualified to make that judgement. Second, and more important, it lets us *version* the rules. A changed configuration is written as a new version with the previous one retired, and each session stores the version that judged it. So if a therapist tightens the squat depth threshold in March, January's reports still reflect the criteria that were actually in force in January. Without that, a patient's recorded progress would change retroactively because the ruler moved.

Q How would you add a sixth exercise?

In the intended case, no Python at all — a seed row defining the required joints, the angle definitions, the state thresholds and the posture rules, added to the exercise library by the idempotent seeding tool. All five current exercises use `BaseExerciseAnalyzer` unchanged. A registry exists so that an exercise needing genuinely novel logic that cannot be expressed declaratively can be given a bespoke class without changing any call site, but the overrides map is empty, which is the state we want it in.

Q Static posture has no repetitions. How does that fit?

As an `ExerciseGoalType` of `HOLD` rather than `REPS`, carried from the database through the analyzer context, so the pose service never has to infer it from the slug. The same analyzer class handles both; what changes is the completion driver — `HoldTracker` accumulating seconds held in correct alignment instead of `MovementMachine` counting cycles. It splits again at report generation, where the denominator becomes seconds prescribed rather than repetitions prescribed, and repetition totals are written as an explicit zero rather than left stale.

SECURITY

Q How does authentication work?

Passwords are hashed with Argon2id at OWASP's recommended parameters. On login the user gets a short-lived JWT access token, held in memory only in the browser rather than in `localStorage` where any injected script could read it, plus an opaque refresh token in an `HttpOnly` cookie scoped to the auth path. The access token is sent as a bearer token; when it expires the client silently refreshes.

Q Why is the refresh token not a JWT?

Because a JWT is self-validating, so a stolen refresh token stays valid until it expires — even after the user logs out. There is nothing to revoke. An opaque random token has to be looked up in the `refresh_tokens` table, so revocation is immediate and reuse is detectable. We store only its SHA-256 hash, so a database leak yields nothing replayable. The same reasoning applies to email-verification and password-reset tokens, which additionally travel in URLs that end up in mail logs and browser history.

Q What stops a patient from faking a perfect session?

The browser is never trusted with a number. To start a session it sends only an assignment id — the exercise, the targets and the rule version are read from the database at creation. The pose service then asks the *backend* for its configuration, never the browser. Repetitions are proposed by the pose service over an internal API authenticated with a service token, and the backend validates and persists them. At the end, the report is computed entirely from rows the backend wrote itself. There is no code path by which a browser-supplied score reaches a report.

Q What is the pose ticket and why not reuse the access token?

It is a separate short-lived JWT carrying `scope: 'pose-session'` and the session id, signed with a different secret from the access token. That means an application access token can never be replayed as a pose ticket, and a pose ticket can never authenticate against the API — two separate secrets and an explicit scope check. It also binds the connection to one specific session rather than to the user in general. Notably, we do *not* re-check expiry per frame: the ticket authorises establishing the connection, and killing a legitimate three-minute exercise because a two-minute ticket lapsed would be hostile. A separate absolute session cap handles the forgotten-socket case.

Q Can a therapist see any patient's data?

No — only patients they are linked to, through a consent-based invite flow with a time-limited code. The checks live in a single `AccessControlService` rather than being scattered across controllers, because one omitted check in one controller is invisible, and it has its own test suite covering exactly that case.

ENGINEERING PRACTICE

Q How did you test this?

305 automated tests across three suites — 74 backend with Jest, 131 on the pose service with pytest, 100 on the frontend with Vitest. They concentrate deliberately on the places where being subtly wrong is worse than crashing: the repetition state machine, the scoring arithmetic, and the access-control rules. A crash is visible; a plausible but wrong repetition count is not. End-to-end backend tests run against an embedded PostgreSQL, so the suite works on a machine with no database installed.

Q Tell us about something that did not work.

The skeleton overlay. To make it look smoother I eased every landmark toward its newest position on each animation frame. The result was worse in exactly the situation that matters: easing toward a target means never quite arriving at one, so while the patient was actually moving the skeleton sat permanently behind them and read as lag. It was reverted to drawing each result exactly, and I dropped a canvas glow effect at the same time — a per-frame GPU cost on a machine already running inference. The file now carries a comment explaining why interpolation must not be reintroduced, because the change looks like an obvious improvement if you have not seen it fail.

Q What would you do differently, and what are the system's real limitations?

The honest limitation is that the thresholds are not clinically validated — they come from the inherited prototype and general biomechanics, not a trial against goniometer ground truth, and validating them is the single most valuable piece of future work. Technically, 2-D pose means angles are measured in the image plane, so movement toward or away from the camera is foreshortened; we mitigate that by prescribing a camera view per exercise, but we do not solve it. And throughput is about three concurrent sessions on a t3.medium. If I started again, I would build the therapist-facing rule editor from the beginning — the data model and the versioning already support it, and without the editing surface the configurability is real but not reachable by the person it was designed for.

Appendix: numbers at a glance

The last page to read before you go in.

3 services	19 database models	52 API endpoints	305 tests, all passing
---------------	-----------------------	---------------------	---------------------------

FACT	VALUE
Services	React SPA (Vercel) · NestJS API · FastAPI pose service · PostgreSQL 16 · Caddy
Lines of application code	~9,200 backend TS · ~9,800 frontend TS/TSX · ~4,600 Python — roughly 23,600 total, plus ~6,400 lines of tests
Database	19 models, 15 enums, 697-line Prisma schema, 4 migrations
API	52 endpoints across 10 controllers, versioned at <code>/api/v1</code> , documented by Swagger
Tests	74 Jest · 131 pytest · 100 Vitest = 305
Exercises	5 — squat, bicep-curl, shoulder-abduction, straight-leg-raise (REPS), static-posture (HOLD)
Pose model	YOLOv8n-pose, 6.8 MB weights, 17 COCO keypoints, ~140 ms/frame on CPU, AGPL-3.0
Frame pipeline	~7 fps · 480 px wide · JPEG q0.6 · 5-frame rolling median
Fault catalogue	19 codes · 3 severities (INFO / WARNING / CRITICAL) · tracking codes separated from form codes
Rep cycle	UP → GOING_DOWN → DOWN → GOING_UP → UP, guarded by hysteresis, dwell, cooldown, visibility gating
Score	frame: 100 – penalties, clamped → rep: mean of frames → session: mean of reps
Rep correctness	score ≥ threshold AND no CRITICAL fault
Auth	Argon2id · 15-min in-memory JWT · opaque SHA-256-hashed refresh token in an HttpOnly cookie · separately-signed, scoped pose ticket
Session lifecycle	CREATED → ACTIVE → FINALIZING → COMPLETED (or CANCELLED / ABANDONED)
Deployment	Vercel + one t3.medium EC2; Caddy is the only host-bound service; Postgres publishes no port at all; 3 concurrent sessions

FIVE SENTENCES TO HAVE READY VERBATIM

1. **The trust rule.** “The browser may state intent; the pose service may propose; only the backend may persist.”
2. **Rep counting.** “A repetition is a completed cycle, not a threshold crossing — guarded by hysteresis, dwell, cooldown and visibility gating.”
3. **The AI question.** “The learning is in the perception layer; the clinical judgement is deliberately rule-based so a physiotherapist can inspect and tune it.”
4. **Rule versioning.** “Each session records the rule version that judged it, so tightening a threshold today cannot retroactively rewrite last month's progress.”
5. **The limitation.** “The thresholds are not clinically validated, and a validation study against goniometer ground truth is the most valuable next step.”